



ASSIGNMENT REPORT

Digital Design

Assignment 5.0 — Linear Feedback Shift Register (LFSR) / CRC Generator

Course	Digital Design
Assignment	Assignment 5.0 — Linear Feedback Shift Register (LFSR) based CRC Generator
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	Eng. Ali Temsah
Submission Date	July 27, 2026

01 OBJECTIVE

This assignment aims to design, implement, and verify a Linear Feedback Shift Register (LFSR) used as a serial CRC generator. The design shifts in one byte of data (LSB first), computes an 8-bit CRC remainder in an 8-bit shift register, and then serially shifts the CRC bits out through a single-bit output port, with a Valid flag marking the CRC transmission window.

02 PROBLEM STATEMENT / REQUIREMENTS

The task is to build an 8-bit Galois-style LFSR/CRC block with the following interface and behaviour, and to verify it with a self-checking testbench:

- Inputs: DATA (serial bit), ACTIVE (data-transmission enable), CLK, RST (asynchronous, active-low).
- Outputs: CRC (serial output bit, registered) and Valid (high only while CRC bits are being shifted out).
- All 8 registers (R7–R0) are asynchronously reset to the seed value 8'hD8.
- DATA bits are shifted in LSB-first while ACTIVE is high; once the byte has been fully shifted in, R7–R0 hold the 8-bit CRC remainder.
- When ACTIVE goes low, the CRC bits are shifted out serially through CRC (R0 first, i.e. LSB first) while Valid is asserted.
- Clock frequency: 10 MHz (100 ns period).
- Testbench reads 10 test vectors from DATA_h.txt and the corresponding expected CRC values from Expec_Out_h.txt, drives the DUT, and self-checks every result.

03 DESIGN & ANALYSIS

3.1 Design Approach

The CRC block is built as an 8-bit Galois LFSR. Two always blocks operate on the same clock/reset: one implements the shift-register datapath (LFSR contents, CRC output, Valid flag) and the other implements a 4-bit Counter used only during the CRC-output phase to know when all 8 CRC bits have been shifted out (Counter_Max = (Counter == 8)).

While ACTIVE is high, every clock edge shifts $\text{DATA_XOR_R0_W1} = \text{DATA} \oplus \text{LFSR}[0]$ into R7, and into every register whose position is marked with a '1' in the tap mask XOR_Bits = 8'b0100_0100 (taps after R6 and after R2), while the remaining registers simply shift toward R0 — this realises the same structure as the Fibonacci/Galois LFSR shown in the assignment's block diagram. Once ACTIVE drops, the design shifts LFSR[0] out onto CRC once per clock (R0 first, R7 last) for 8 cycles, raising Valid for that window and feeding 0 into R7 from then on.

3.2 Register / Tap Configuration

Instead of a truth table, the relevant design constants are summarised below, since the block behaves as a shift register rather than combinational logic:

Parameter	Value
No_Registers	8
SEED (reset value)	8'hD8 (1101_1000)
XOR_Bits (feedback taps)	8'b0100_0100 → taps after R6 and R2
Shift-in order	DATA, LSB first, into R7 side
Shift-out order	CRC bits, R0 first (LSB) → R7 last (MSB)

3.3 Circuit / Block Diagram

Block interface of the CRC module, as given in the assignment:

Block Interface

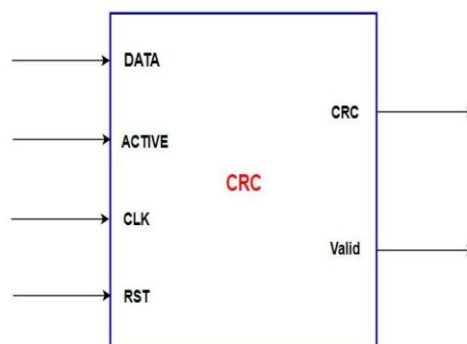


Figure 1: CRC block interface — DATA, ACTIVE, CLK, RST in; CRC, Valid out.

Internal LFSR structure, showing the two feedback XOR gates tapped after R7 and R3 (matching XOR_Bits = 0100_0100):

Block Diagram

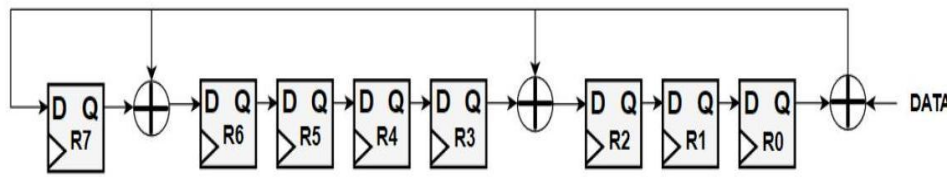


Figure 2: 8-bit Galois LFSR block diagram used for the CRC computation.

3.4 HDL Implementation

The design is implemented in Verilog as module CRC (file CRC.v). The core feedback/shift datapath is shown below:

```
// Port Declaration of CRC Module
module CRC (
    input DATA, ACTIVE,
    input CLK, RST,
    // Serial CRC Output
    output reg CRC,
    // Check The Output is Valid to See or No
    output reg Valid
);
    localparam [3:0] No_Registers = 8; // Set The Number of the registers
    localparam [No_Registers - 1:0] SEED = 'hD8; // RST Default Value for the
Register
    localparam [No_Registers - 1:0] XOR_Bits = 'b0100_0100; // Set the XOR
Gates Places

    reg [No_Registers - 1:0] LFSR; // Registers for the algorithm and shift
register
    reg [3:0] Counter; // Counts output bits shifted so far
    integer i;

    wire DATA_XOR_R0_W1; // Feedback signal
    wire Counter_Max; // Finished shifting out all CRC bits

    assign Counter_Max = (Counter == No_Registers);
    assign DATA_XOR_R0_W1 = DATA ^ LFSR[0];

    // Outputs and registers setting
    always @(posedge CLK or negedge RST) begin
        if (!RST) begin
            LFSR <= SEED;
            Valid <= 0;
        end
        else if (ACTIVE) begin
            Valid <= 0;
        end
    end
endmodule
```

```

        LFSR[No_Registers - 1] <= DATA_XOR_R0_W1;
    for (i = 0; i < No_Registers - 1; i = i + 1) begin
        if (XOR_Bits[i])
            LFSR[i] <= LFSR[i + 1] ^ DATA_XOR_R0_W1;
        else
            LFSR[i] <= LFSR[i + 1];
        end
    end
else if (!Counter_Max) begin
    Valid <= 1;
    CRC <= LFSR[0];
    LFSR[No_Registers - 1] <= 0;
    for (i = 0; i < No_Registers - 1; i = i + 1) begin
        LFSR[i] <= LFSR[i + 1];
    end
end
else begin
    Valid <= 0;
    CRC <= 0;
end
end
end

// Setting the Counter
always @(posedge CLK or negedge RST) begin
    if (!RST) begin
        Counter <= No_Registers;
    end
    else if (ACTIVE) begin
        Counter <= 0;
    end
    else if (!Counter_Max) begin
        Counter <= Counter + 1;
    end
end
endmodule //CRC

```

04 SIMULATION & RESULTS

The design was verified in ModelSim using a self-checking testbench (CRC_TB.v) driven by a 10 MHz clock (100 ns period). The testbench reads 10 DATA bytes and their expected CRC values from DATA_h.txt / Expec_Out_h.txt, resets the DUT, drives each byte in LSB-first with ACTIVE high, waits for Valid to assert, samples 8 CRC bits (R0 first), and compares the result against the expected value, tallying passes and failures.

Key testbench tasks:

```

`timescale 1ns/1ps
module CRC_TB;
    //-----
    -

```

```

// Parameters
//-----
-
localparam CLK_PERIOD    = 100; // 10 MHz -> 100 ns period
localparam No_Registers = 8;    // No of registers in the algorithm
localparam No_Tests     = 10;   // no of test cases

reg [No_Registers - 1:0] DATA_input      [0:No_Tests - 1];
reg [No_Registers - 1:0] Expected_Out_CRC [0:No_Tests - 1];

//-----
-

// Signals
//-----
-

reg  DATA_TB, ACTIVE_TB, CLK_TB, RST_TB;
wire CRC_TB, Valid_TB;

integer test_num = 0;
integer passed   = 0;
integer failed   = 0;

//-----
-

// DUT Instantiation
//-----
-

CRC DUT_CRC (
    .DATA(DATA_TB), .ACTIVE(ACTIVE_TB),
    .CLK(CLK_TB),   .RST(RST_TB),
    .CRC(CRC_TB),   .Valid(Valid_TB)
);

//-----
-

// Clock Generation: 10 MHz (100 ns period)
//-----
-

initial begin
    CLK_TB = 0;
    forever #(CLK_PERIOD / 2) CLK_TB = ~CLK_TB;
end

//-----
-

// Timeout Watchdog
//-----
-

```

```

initial begin
    #100000;
    $display("[TIMEOUT] Simulation exceeded limit. Terminating.");
    $finish;
end

//-----
-
// Tasks
//-----
-

task do_reset;
    begin
        RST_TB    = 1'b0;
        ACTIVE_TB = 1'b0;
        DATA_TB  = 1'b0;
        @(negedge CLK_TB);
        @(negedge CLK_TB);
        RST_TB = 1'b1;
        @(negedge CLK_TB);
    end
endtask

task drive_byte;
    input [No_Registers-1:0] byte_in;
    integer i;
    begin
        ACTIVE_TB = 1'b1;
        for (i = 0; i < No_Registers; i = i + 1) begin
            DATA_TB = byte_in[i];    // LSB first
            @(negedge CLK_TB);
        end
        ACTIVE_TB = 1'b0;
    end
endtask

task capture_and_check;
    input [No_Registers-1:0] expected;
    input integer idx;
    reg [No_Registers-1:0] received;
    integer i;
    begin
        @(posedge Valid_TB);
        for (i = 0; i < No_Registers; i = i + 1) begin
            @(negedge CLK_TB);
            received[i] = CRC_TB;    // R0 first -> LSB, R7 Last -> MSB
        end
    end
endtask

```

```

        if (received === expected) begin
            $display("Test %0d PASSED (expected=%h, got=%h)", idx,
expected, received);
            passed = passed + 1;
        end else begin
            $display("Test %0d FAILED (expected=%h, got=%h)", idx,
expected, received);
            failed = failed + 1;
        end
    end
endtask

//-----
-
// Main stimulus
//-----
-

initial begin
    $dumpfile("CRC_DUMP.vcd");
    $dumpvars;

    $readmemh("DATA_h.txt", DATA_input);
    $readmemh("Expec_Out_h.txt", Expected_Out_CRC);

    for (test_num = 0; test_num < No_Tests; test_num = test_num + 1) begin
        do_reset;
        drive_byte(DATA_input[test_num]);
        capture_and_check(Expected_Out_CRC[test_num], test_num);
    end

    $display("-----");
    $display("TOTAL: %0d passed, %0d failed out of %0d", passed, failed,
No_Tests);
    $display("-----");
    #(CLK_PERIOD);
    $finish;
end

endmodule //CRC_TB

```

Simulation results for all 10 test vectors:

Test #	Input DATA byte (bin / hex)	Expected CRC (hex)	Simulated CRC (hex)	Result
0	10010011 (0x93)	0x78	0x78	PASS
1	01110010 (0x72)	0x44	0x44	PASS
2	00110110 (0x36)	0x11	0x11	PASS
3	00011011 (0x1B)	0xd2	0xd2	PASS
4	10100110 (0xA6)	0x09	0x09	PASS
5	11000000 (0xC0)	0xb2	0xb2	PASS
6	01010101 (0x55)	0x36	0x36	PASS
7	11110010 (0xF2)	0x80	0x80	PASS
8	01011110 (0x5E)	0x2c	0x2c	PASS

Test #	Input DATA byte (bin / hex)	Expected CRC (hex)	Simulated CRC (hex)	Result
9	00010001 (0x11)	0x63	0x63	PASS

Console summary from the ModelSim transcript:

```
# Test 0 PASSED (expected=78, got=78)
# Test 1 PASSED (expected=44, got=44)
# Test 2 PASSED (expected=11, got=11)
# Test 3 PASSED (expected=d2, got=d2)
# Test 4 PASSED (expected=09, got=09)
# Test 5 PASSED (expected=b2, got=b2)
# Test 6 PASSED (expected=36, got=36)
# Test 7 PASSED (expected=80, got=80)
# Test 8 PASSED (expected=2c, got=2c)
# Test 9 PASSED (expected=63, got=63)
#
# -----
# TOTAL: 10 passed, 0 failed out of 10
#
# ** Note: $finish : [REDACTED] /CRC_TB.v(119)
# Time: 19100 ns Iteration: 0 Instance: /CRC_TB
```

Waveform snapshots from ModelSim, showing DATA being shifted in and the CRC/Valid outputs being shifted out for the first test cases:

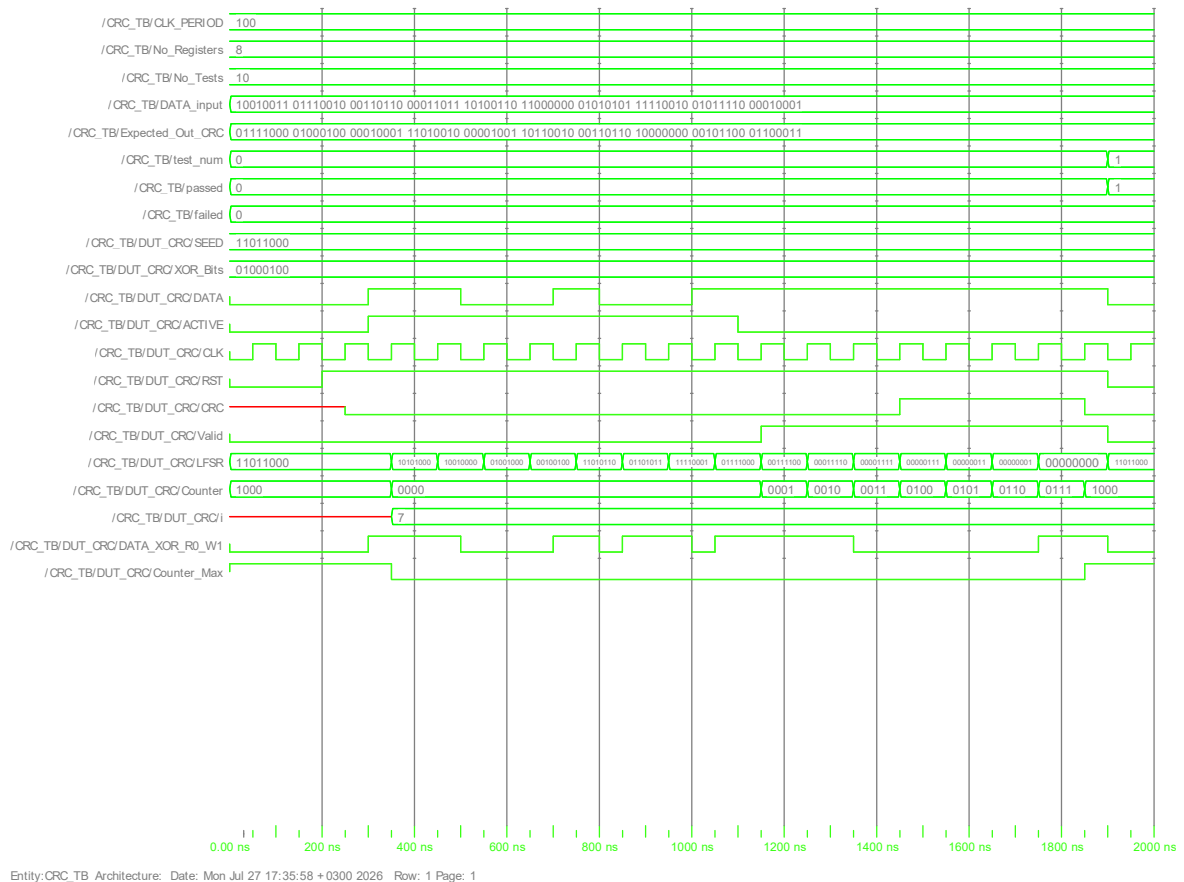


Figure 3: Waveform for Test 0 — DATA_input[0]=8'h93, LFSR reset to SEED (D8), CRC/Counter progressing through the shift-out phase.

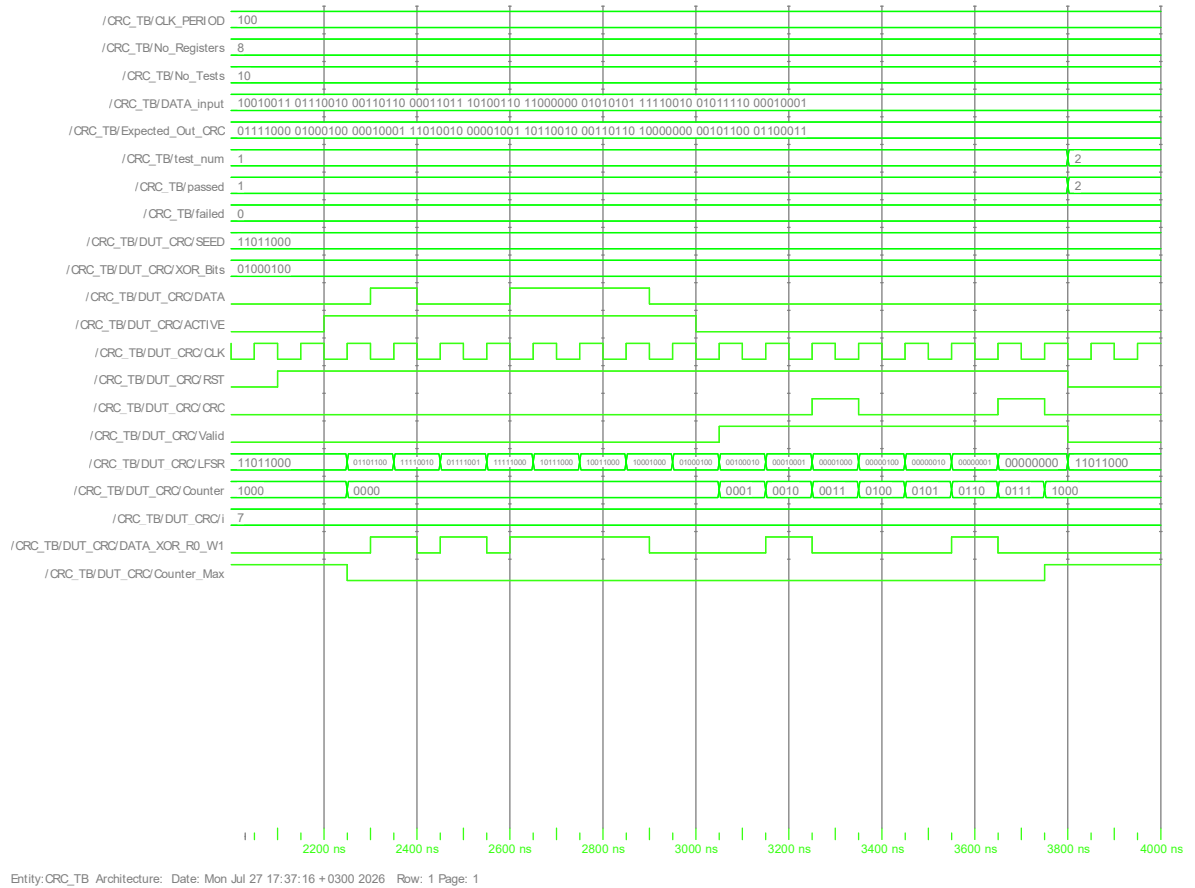


Figure 4: Waveform for Test 1 — LFSR content after loading DATA_input[1]=8'h72, matching Expected_Out_CRC[1]=0x44.

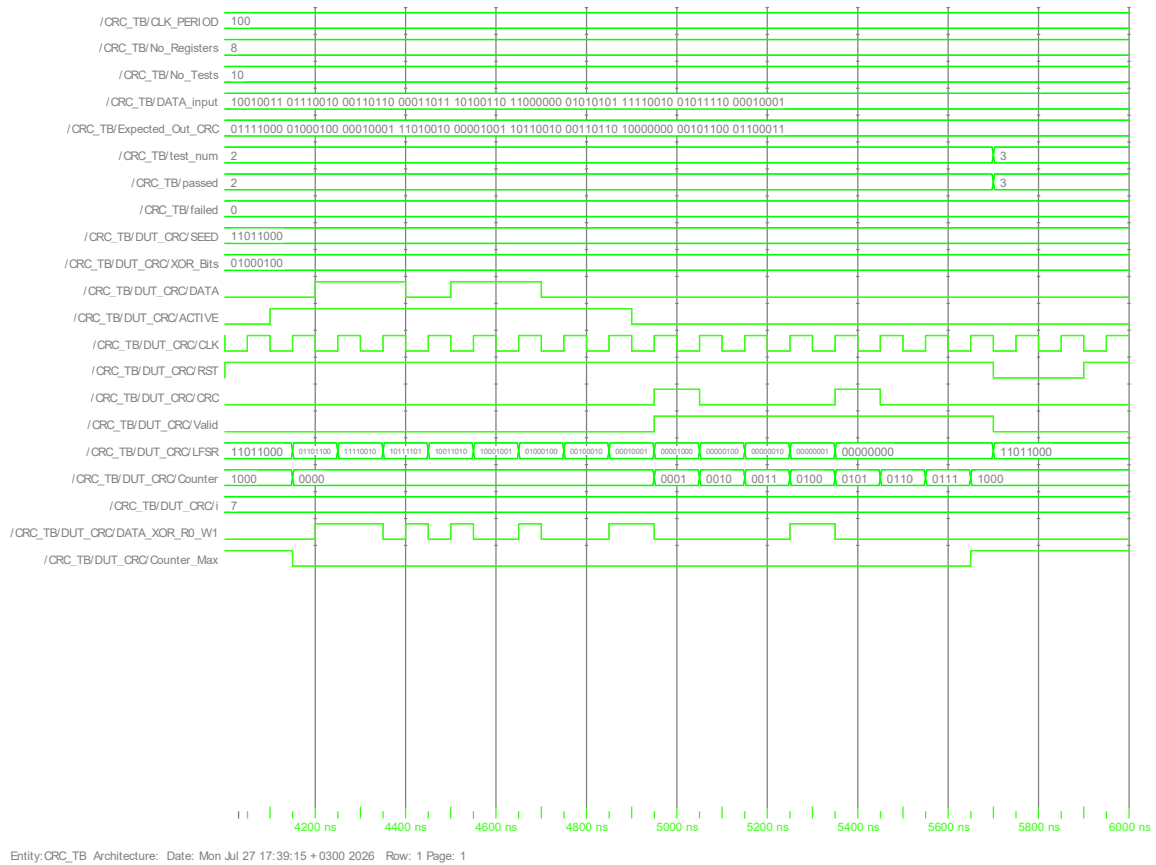


Figure 5: Waveform for Test 2 — passed/failed counters incrementing (passed=2, failed=0) as successive tests complete.

05 DISCUSSION

All 10 test cases passed with the simulated CRC output matching the expected value exactly (10 passed, 0 failed), confirming that the LFSR correctly implements the specified feedback polynomial (taps after R6 and R2) and that the shift-in / shift-out ordering (DATA LSB-first in, CRC R0-first out) matches the specification.

One subtlety worth noting is the timing of Valid and Counter: because Counter is only re-evaluated on the clock edge after ACTIVE deasserts, the first CRC bit becomes valid one cycle after ACTIVE goes low, which the testbench correctly handles by waiting on the positive edge of Valid before sampling. No timing violations, race conditions, or mismatches were observed across the 10 vectors, and the asynchronous active-low reset correctly re-initialised the registers to the seed (8'hD8) before every test.

06 CONCLUSION

An 8-bit Galois LFSR-based CRC generator was designed in Verilog, matching the specified seed, feedback taps, and serial shift-in/shift-out protocol, and was verified against 10 self-checking test vectors in ModelSim with a 100% pass rate. The exercise reinforced how LFSR feedback taps determine the generator polynomial, how registered outputs affect output timing relative to a Valid flag, and how a file-driven, self-checking testbench simplifies regression-style verification of serial protocols.